

UD5: DISEÑO ORIENTADO A OBJETOS. DIAGRAMAS DE CLASE.

ÍNDICE

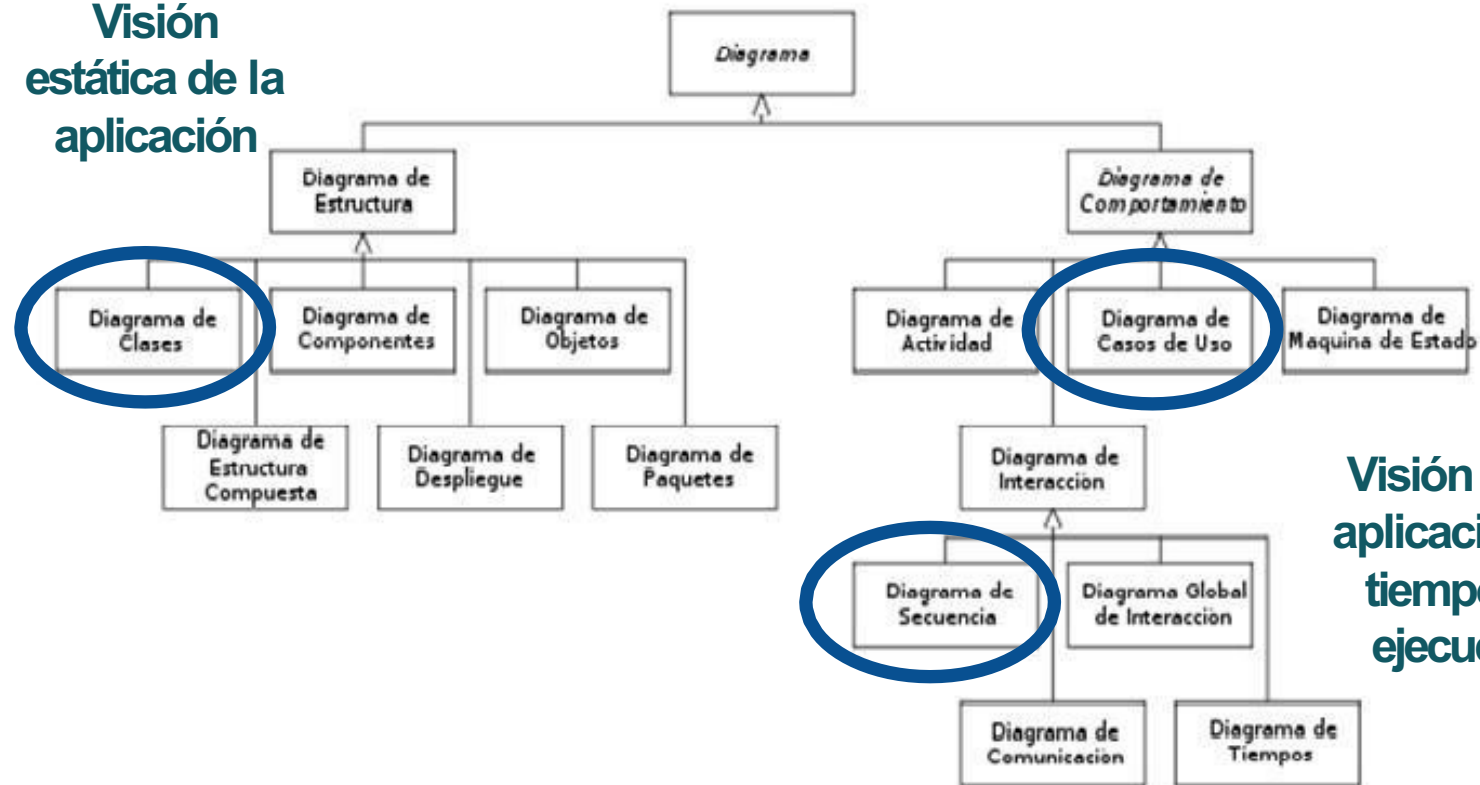
- Introducción a UML
- Herramientas para la elaboración de diagramas UML
- Diagramas de clases en UML
- Generación de código a partir de diagramas de clases (Ingeniería directa)
- Ingeniería inversa
- Generación de documentación

I. INTRODUCCIÓN A UML

- **UML** (*Unified Modeling Language* o *Lenguaje Unificado de Modelado*): Conjunto unificado de estándares para las diferentes necesidades y usos que un diseñador pudiera tener a la hora de plantear una representación gráfica de un programa.
- Es un lenguaje que permite modelar, construir y documentar los elementos que forman un sistema software orientado a objetos.
- UML 2.4.1 es la última versión y es mantenida por la entidad OMG (Object Management Group), que deciden las características y notaciones del lenguaje.

I. INTRODUCCIÓN A UML

Visión
estática de la
aplicación



Visión de la
aplicación en
tiempo de
ejecución

I. INTRODUCCIÓN A UML

- Teniendo en cuenta la superestructura de diagramas UML que se pueden realizar (slide anterior), podríamos pensar que la tarea es abrumadora.
- En la práctica, lo más común es implementar diagramas de clases, de secuencia y de casos de uso.

“UML permite a los desarrollador@s visualizar el producto de su trabajo en esquemas o diagramas estandarizados denominados modelos que representan el sistema desde diferentes perspectivas”

I. INTRODUCCIÓN A UML

- Ventajas del modelado en UML:
 - Porque permite utilizar un *lenguaje común que facilita la comunicación* entre el equipo de desarrollo.
 - Con UML podemos *documentar* todos los componentes de un proyecto de desarrollo, por lo que se dispone de documentación que trasciende al proyecto.
 - Hay *estructuras que trascienden lo representable en un lenguaje de programación*, como las que hacen referencia a la arquitectura del sistema, relaciones entre módulos, o en qué nodos hardware se ejecutarán cuando trabajamos con sistemas distribuidos.
 - Permite especificar todas las decisiones de análisis, diseño e implementación, construyéndose modelos precisos, no ambiguos y completos. (...).
 - Además UML puede conectarse a lenguajes de programación concretos por *ingeniería directa e inversa*, como veremos más adelante.

2. HERRAMIENTAS PARA LA ELABORACIÓN DE DIAGRAMAS UML

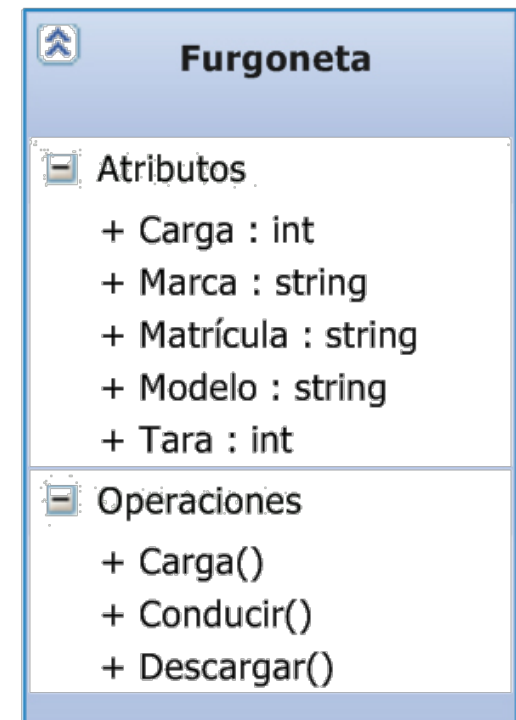
- Podemos acceder a herramientas CASE que facilitan en gran manera el desarrollo de los diagramas UML.
- Estas herramientas suelen contar con un entorno de ventanas tipo wysiwyg (**W**hat **Y**ou **S**ee **I**s **W**hat **Y**ou **G**et), permiten documentar los diagramas e integrarse con otros entornos de desarrollo incluyendo la generación automática de código y procedimientos de ingeniería inversa:
 - Rational Systems Developer de IBM: Herramienta propietaria que permite el desarrollo de proyectos software basados en la metodología UML.
 - Visual Paradigm for UML (VP-UML): Incluye una versión para uso no comercial que se distribuye libremente sin más que registrarse para obtener un archivo de licencia. Incluye diferentes módulos para realizar desarrollo UML, diseñar bases de datos, realizar actividades de ingeniería inversa y diseñar con Agile. Es compatible con diferentes IDEs. Multiplataforma.
 - Otros: UML Designer (Eclipse plugin), Enterprise Architect, etc.

2. DIAGRAMAS DE CLASES UML

- Es el tipo de diagrama más importante porque representa los elementos estáticos del sistema, sus atributos y comportamientos, y como se relacionan entre ellos.
- Contiene las clases del dominio del problema, y a partir de éste se obtendrán las clases que formarán después el programa informático que dará solución al problema.

2. DIAGRAMAS DE CLASES UML

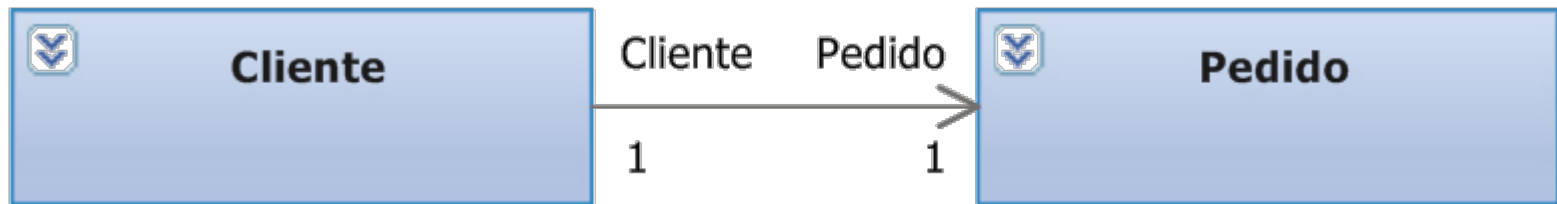
- En un diagrama de clases podemos encontrar:
 1. Clases:
 - Cada bloque dentro del diagrama representa a una clase.
 - Una clase, como sabemos, dispone de unos atributos y de unos métodos asociados. Se indica su visibilidad.
 - Representaríamos las clases como cajas en donde escribiríamos sus atributos y sus métodos.



2. DIAGRAMAS DE CLASES UML

2. Relaciones:

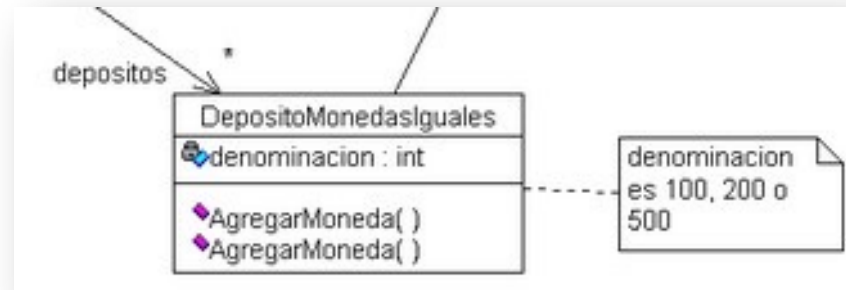
- Representan relaciones reales entre los elementos del sistema a los que hacen referencia las clases.
- Pueden ser de asociación, agregación y herencia.
- Por ejemplo si tengo una clase persona, puedo establecer una relación conduce entre persona y coche.



2. DIAGRAMAS DE CLASES UML

3. Notas:

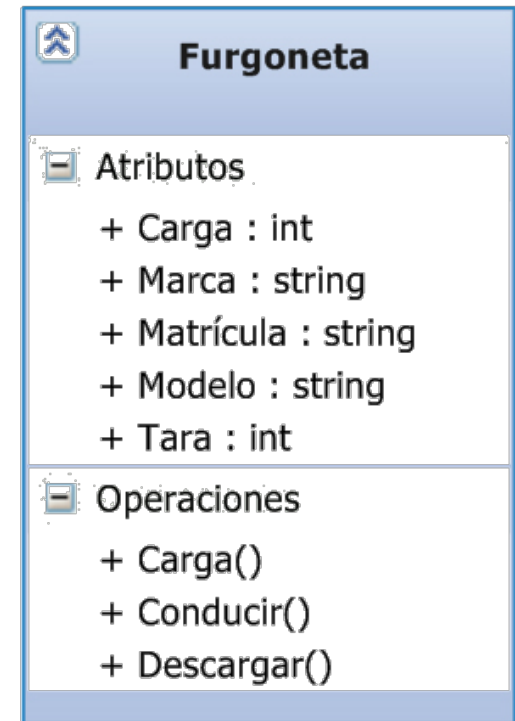
- Se representan como un cuadro donde podemos escribir comentarios que nos ayuden a entender algún concepto que queramos representar.



2. DIAGRAMAS DE CLASES UML

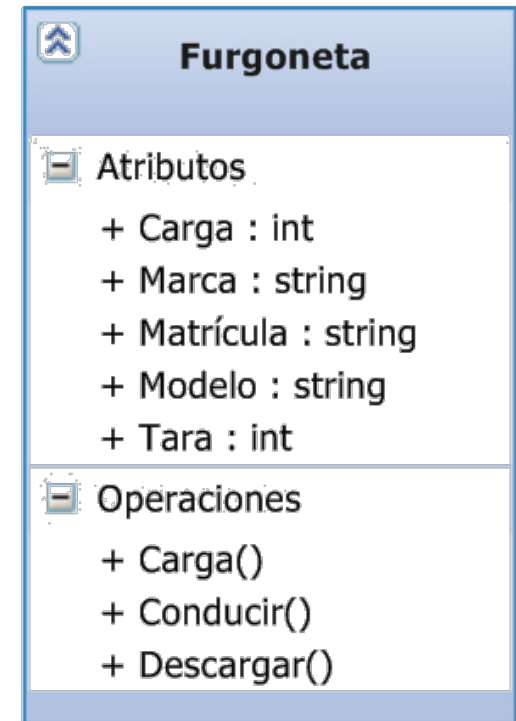
➤ Notación:

- Las clases se definen mediante cuadrados, y dichos cuadrados se dividen en tres segmentos horizontales.
- Primero tendríamos el nombre de la clase, seguidamente irían los atributos y, al final, los métodos de la clase.
- Es importante resaltar que si alguna clase no posee atributos propios o métodos propios, se deberá seguir dibujando el segmento que le corresponde, incluso si está vacío.



2. DIAGRAMAS DE CLASES UML

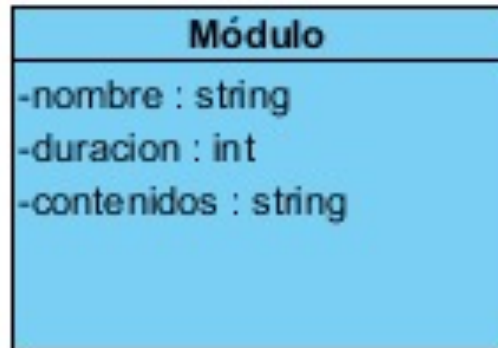
- El carácter que precede al nombre del atributo o método:
 - **(+)** **Público**: el elemento será visible tanto desde dentro como desde fuera de la clase.
 - **(-)** **Privado**: el elemento de la clase solo será visible desde la propia clase.
 - **(#)** **Protegido**: el elemento no será accesible desde fuera de la clase, pero podrá ser manipulado por los demás métodos de la clase o de las subclases.
- Después del método o atributo, tendríamos un tipo de datos de devolución separado por “:”.
- En el caso de los métodos, entre paréntesis vendrían los tipos de datos que espera el método en cuestión.



2. DIAGRAMAS DE CLASES UML

A5.2. Crear una clase de nombre "Módulo" y que tenga tres atributos:

- Nombre, de tipo string.
- Duración de tipo int.
- Contenidos de tipo string.

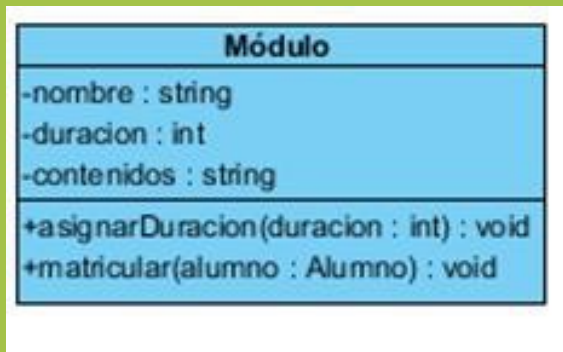


2. DIAGRAMAS DE CLASES UML



A5.3. Añadir a la clase creada anteriormente los métodos:

- matricular(alumno : Alumno) : void
- asignarDuración(duracion : int) : void



2. DIAGRAMAS DE CLASES UML

➤ Relaciones:

- En un diagrama de clases, además de dibujar y representar las clases con sus atributos y métodos, también se representan las relaciones.
- Las relaciones **se representan con flechas** con una forma determinada, **y, además, poseen una cardinalidad.**
- La cardinalidad es un número o símbolo que representa al número de elementos de cada clase en cada relación, pudiendo usar el comodín “*” para definir un número indeterminado de elementos.

Cardinalidad	Significado
1	Uno y solo uno
0..1	Cero o uno
X..Y	Desde X hasta Y
*	Cero o varios
0..*	Cero o varios
1..*	Uno o varios

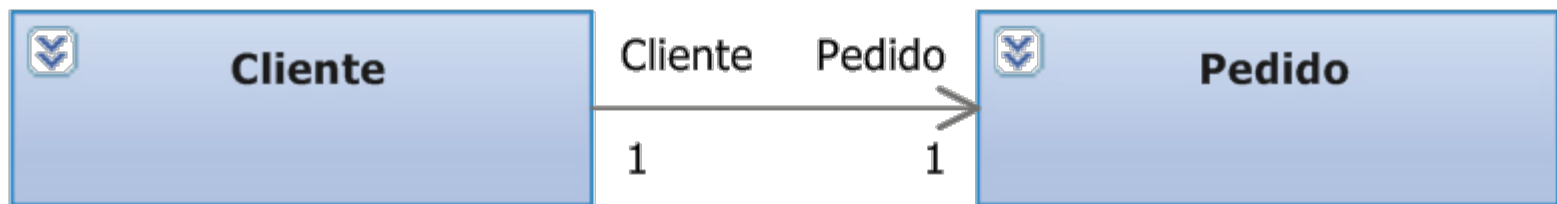
2. DIAGRAMAS DE CLASES UML

➤ Relaciones:

➤ Tipos de relaciones:

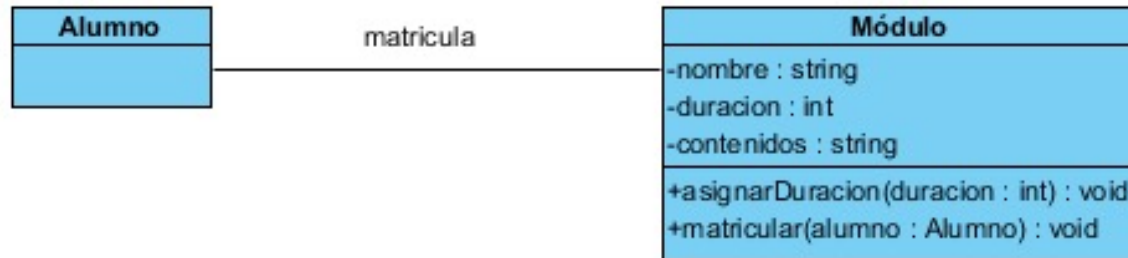
1. Asociación:

- La asociación es la más básica de las relaciones, no tiene un tipo definido y puede ser tanto una composición como una agregación, además una asociación podría implicar únicamente un uso del objeto asociado.
- Se representan mediante una flecha simple que también puede tener una cardinalidad.



2. DIAGRAMAS DE CLASES UML

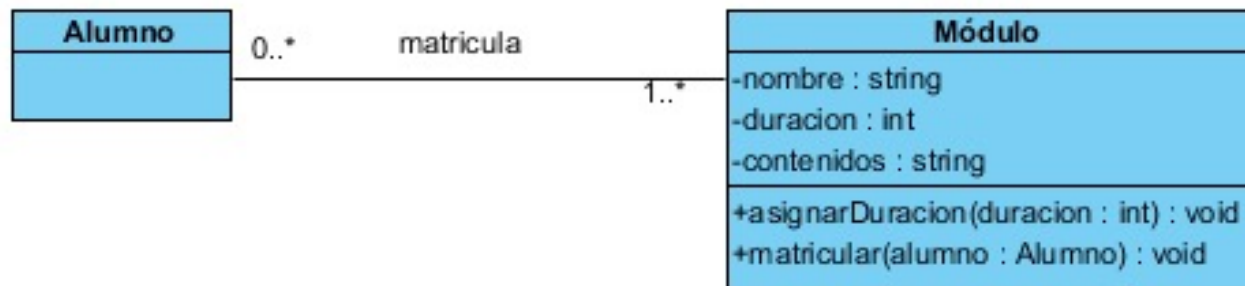
A5.4. Crea una clase nueva llamada Alumno y establece una relación de asociación con el nombre “matrícula” entre ésta y la clase Módulo



2. DIAGRAMAS DE CLASES UML



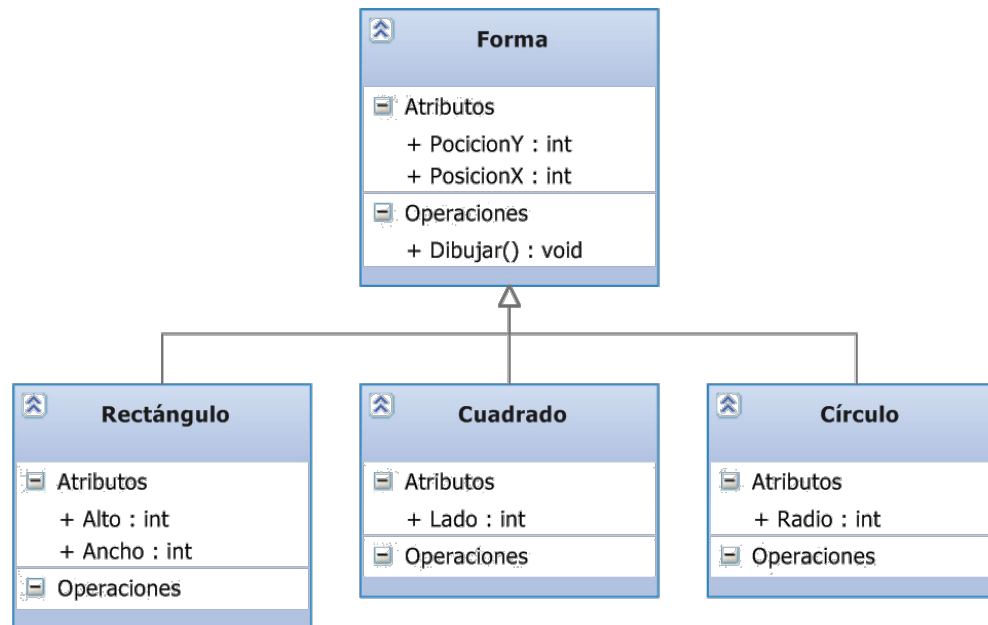
A5.5. Establece la cardinalidad de la relación que has creado en el punto anterior para indicar que un alumno debe estar matriculado en al menos un módulo, o varios y que para cada módulo se puede tener ningún alumno, uno o varios.



2. DIAGRAMAS DE CLASES UML

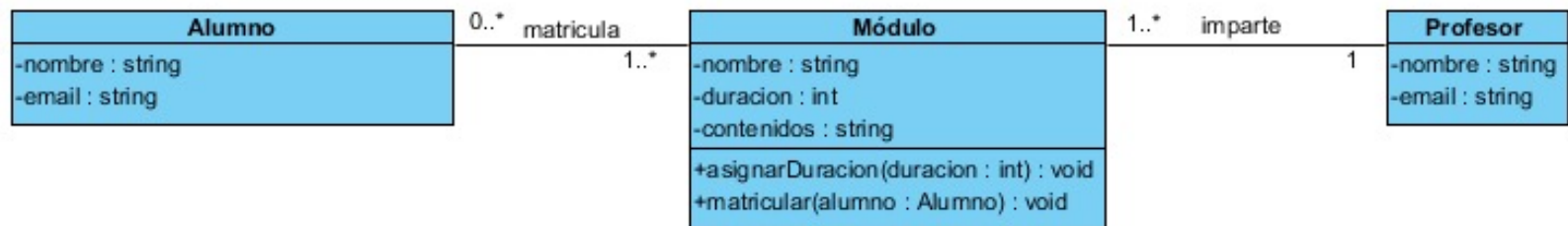
Herencia:

- La herencia es un modo de representar clases y subclases, es decir, clases más específicas de una general.
- Se representan mediante una flecha con una punta triangular vacía.



2. DIAGRAMAS DE CLASES UML

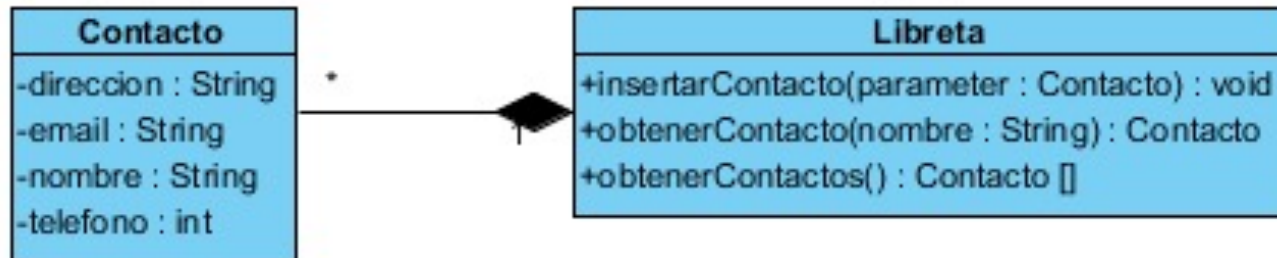
A5.6. En el diagrama que se muestra a continuación, tenemos Alumnos y Profesores. Aún no hemos hablado de su definición y estructura, pero en nuestro sistema tanto un alumno como un profesor tienen unas características comunes como el nombre o el correo electrónico por el hecho de ser personas. Transforma este diagrama para hacer uso de la herencia añadiendo una clase "Persona".



2. DIAGRAMAS DE CLASES UML

Composición:

- La composición define los componentes de los que se compone otra clase.
- La clase que contiene la composición no tiene sentido de existencia si la(s) agregada(s) desaparece(n).
- Mediante esta relación, denotada por una *flecha con un rombo relleno en una de sus puntas*, se indica la presencia de las clases origen en la clase destino, donde la clase destino es apuntada por el rombo de la relación.



2. DIAGRAMAS DE CLASES UML

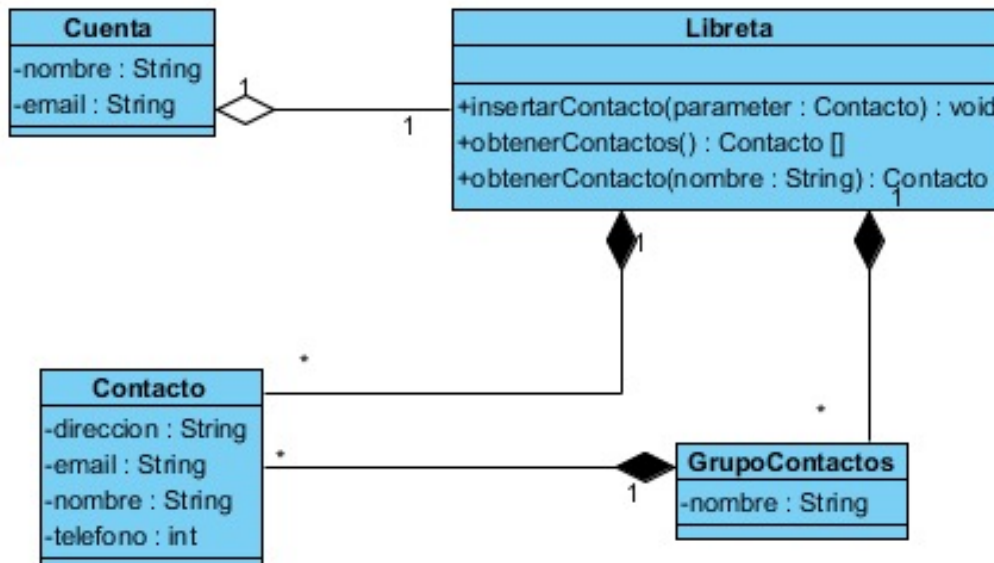


A5.7. Crea un diagrama de clases en base al anterior y modifícala, para poder añadir además grupos de contactos. Los grupos de contactos contendrán contactos y la libreta contendrá tanto grupos de contactos como contactos. Los contactos, además, no necesitan estar en un grupo de contactos para estar en la libreta.

2. DIAGRAMAS DE CLASES UML

Agregación:

- (pertenece a, tiene un, es parte de...)
- Nuestra libreta sigue complicándose, ya que ahora se define la posibilidad de tener una cuenta de usuario para acceder a nuestra libreta de contactos. Las cuentas tendrán un nombre de cuenta y un email, y podrán acceder a la libreta de direcciones. Para representar la relación de esta nueva clase, vamos a utilizar una agregación.



2. DIAGRAMAS DE CLASES UML

- Diferencia principal con la composición reside en que la agregación no implica que la clase agregada necesite una instancia de la otra clase, es decir, la Cuenta no necesita tener una instancia de la clase Libreta y Libreta tampoco una instancia de la clase Cuenta.
- Sólo implica que la clase Cuenta accede a la clase Libreta.

2. DIAGRAMAS DE CLASES UML

Atributos de enlace:

- Es posible que tengamos alguna relación en la que sea necesario añadir algún tipo de información que lo complete de alguna manera.
- Cuando esto ocurre podemos añadir atributos a la relación.
- Ver siguiente ejercicio.

2. DIAGRAMAS DE CLASES UML



A5.8. Volviendo al ejemplo del Alumno, cuando un alumno **se matricula** de un módulo es preciso especificar por ejemplo el curso al que pertenece la matrícula o la calificación final obtenida. Estas características no pertenecen totalmente al alumno ni al módulo sino a la relación específica que se crea entre ellos, que además será diferente si cambia el alumno o el módulo. Añade estos atributos al enlace entre Alumno y Módulo.

Para modelar esto en Visual Paradigm creamos una clase nueva (Matrícula) junto a Alumno y Módulo, y la unimos a la relación utilizando el icono de la paleta "Association class".

2. DIAGRAMAS DE CLASES UML



A5.9. Realizar en Visual Paradigm el diagrama de clases de la página 140 del libro (Fig. 5.10)

3. GENERACIÓN DE CÓDIGO A PARTIR DE DIAGRAMAS DE CLASES

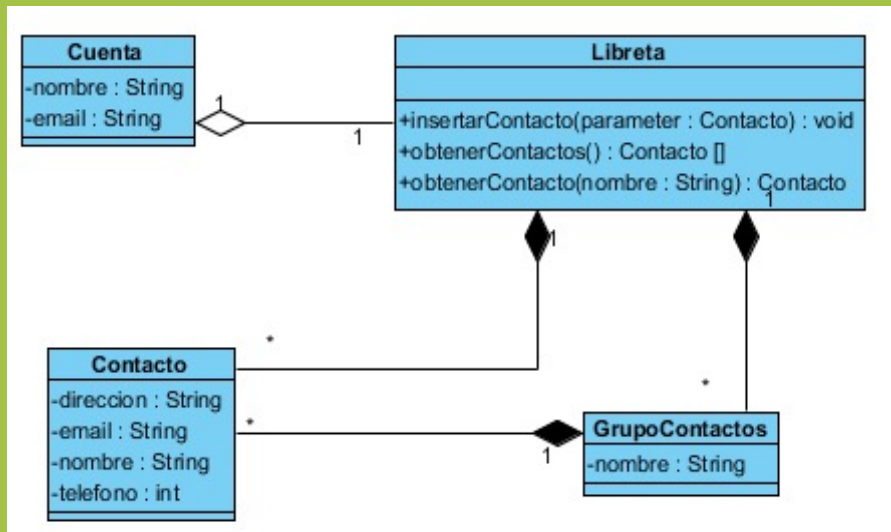
- La Generación Automática de Código consiste en la creación utilizando herramientas CASE de código fuente de manera automatizada.
- El proceso pasa por establecer una correspondencia entre los elementos formales de los diagramas y las estructuras de un lenguaje de programación concreto.
- El diagrama de clases es un buen punto de partida porque permite una traducción bastante directa de las clases representadas gráficamente, a clases escritas en un lenguaje de programación específico como Java o C#.
- Normalmente las herramientas de generación de diagramas UML incluyen la facilidad de la generación, o actualización automática de código fuente, a partir de los diagramas creados.

3. GENERACIÓN DE CÓDIGO A PARTIR DE DIAGRAMAS DE CLASES

- El lenguaje final de implementación de la aplicación influyen en algunas decisiones a tomar cuando estamos creando el diagrama ya que el proceso de traducción es inmediato.
- Si existe algún problema en los nombres de clases, atributos o tipos de datos porque no puedan ser utilizados en el lenguaje final o no existan la generación dará un fallo y no se realizará.
- *Por ejemplo, si queremos utilizar la herramienta de generación de código tendremos que asegurarnos de utilizar tipos de datos simples apropiados, es decir, si usamos Java el tipo de dato para las cadenas de caracteres será String en lugar de string o char*.*

3. GENERACIÓN DE CÓDIGO A PARTIR DE DIAGRAMAS DE CLASES

A5.10. Modifica el diagrama de la aplicación de la Libreta, para que coincida con el siguiente diagrama, y a continuación genera el código asociado al mismo pinchando en el botón “Update code”. Cambia a la perspectiva Java, y verás cómo se han generado los fuentes (INGENIERÍA DIRECTA).



3. INGENIERÍA INVERSA

- La ingeniería inversa se define como el proceso de analizar código, documentación y comportamiento de una aplicación para identificar sus componentes actuales y sus dependencias y para extraer y crear una abstracción del sistema e información del diseño.
- El sistema en estudio no es alterado, sino que se produce un conocimiento adicional del mismo.

3. INGENIERÍA INVERSA



A5.11. Sobre el proyecto del ejercicio anterior A5.10, abrir la perspectiva Java, y añadir un método `toString()` que devuelva una cadena de caracteres con los valores de los atributos. Después seleccionando el proyecto, pinchar en “Update UML Model”. Ir a la vista de modelado y abrir el diagrama de clases para ver que éste se ha actualizado con el nuevo método introducido.

3. INGENIERÍA INVERSA



A5.12. Ahora crea un nuevo proyecto Java y añade, los *.java que se han generado en el proyecto anterior.:

- 1) Sobre el la carpeta src, botón derecho y dar al botón “Update UML Model”.
- 2) Seleccinando de nuevo la carpeta src, dar a Modeling→RoundTrip→Reverse code, seleccionamos la opción Java. En Source path, seleccionar la carpeta src del proyecto, y en Model dejar <Root>.
- 3) Cambiar la perspectiva a Modeling, y seleccionar el “Class repository”. Ahí veremos una carpeta Java Round Trip.
- 4) Reverse “src” to New Class Diagram.
Ver el diagrama que ha generado.